

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include "Pilha.h"
4
5  void criarPilha(EstruturaPilha* estrutura) {
6      estrutura->topoAtual = NULL;
7  }
8
9  int pilhaVazia(EstruturaPilha* estrutura) {
10     return (estrutura->topoAtual == NULL);
11 }
12
13 void empilhar(EstruturaPilha* estrutura, int numero) {
14     CelulaPilha* novoNo = (CelulaPilha*)malloc(sizeof(CelulaPilha));
15     if (novoNo == NULL) return;
16     novoNo->dado = numero;
17     novoNo->anterior = estrutura->topoAtual;
18     estrutura->topoAtual = novoNo;
19 }
20
21 int desempilhar(EstruturaPilha* estrutura) {
22     if (pilhaVazia(estrutura)) return -1;
23     CelulaPilha* auxiliar = estrutura->topoAtual;
24     int valor = auxiliar->dado;
25     estrutura->topoAtual = auxiliar->anterior;
26     free(auxiliar);
27     return valor;
28 }
29
30 void mostrarPilha(EstruturaPilha* estrutura) {
31     if (pilhaVazia(estrutura)) {
32         printf("[Pilha Vazia]\n");
33         return;
34     }
35     EstruturaPilha temp;
36     criarPilha(&temp);
37     while (!pilhaVazia(estrutura)) {
38         int v = desempilhar(estrutura);
39         printf("%d\n", v);
40         empilhar(&temp, v);
41     }
42     while (!pilhaVazia(&temp)) {
43         empilhar(estrutura, desempilhar(&temp));
44     }
45 }
46
47 void liberarPilha(EstruturaPilha* estrutura) {
48     while (!pilhaVazia(estrutura)) {
49         desempilhar(estrutura);
50     }
51 }
52
53 int buscarElemento(EstruturaPilha* estrutura, int numero) {
54     EstruturaPilha temp;
55     criarPilha(&temp);
56     int posicao = 1;
57     int encontrado = -1;
58     while (!pilhaVazia(estrutura)) {
59         int v = desempilhar(estrutura);
60         if (v == numero && encontrado == -1) {
61             encontrado = posicao;
62         }
63         empilhar(&temp, v);
64         posicao++;
65     }
66     while (!pilhaVazia(&temp)) {

```

```

67     empilhar(estrutura, desempilhar(&temp));
68 }
69 return encontrado;
70 }
71
72 int buscarERemover(EstruturaPilha* estrutura, int numero) {
73     EstruturaPilha temp;
74     criarPilha(&temp);
75     int encontrado = 0;
76     while (!pilhaVazia(estrutura)) {
77         int v = desempilhar(estrutura);
78         if (v == numero && !encontrado) {
79             encontrado = 1;
80         } else {
81             empilhar(&temp, v);
82         }
83     }
84     while (!pilhaVazia(&temp)) {
85         empilhar(estrutura, desempilhar(&temp));
86     }
87     return encontrado;
88 }
89
90 void removerPares(EstruturaPilha* estrutura) {
91     EstruturaPilha temp;
92     criarPilha(&temp);
93     while (!pilhaVazia(estrutura)) {
94         int v = desempilhar(estrutura);
95         if (v % 2 != 0) {
96             empilhar(&temp, v);
97         }
98     }
99     while (!pilhaVazia(&temp)) {
100         empilhar(estrutura, desempilhar(&temp));
101     }
102 }
103
104 void removerRepetidos(EstruturaPilha* estrutura) {
105     EstruturaPilha tempLimpa;
106     EstruturaPilha tempVerificadora;
107     criarPilha(&tempLimpa);
108     criarPilha(&tempVerificadora);
109     while (!pilhaVazia(estrutura)) {
110         int atual = desempilhar(estrutura);
111         int duplicado = 0;
112         while (!pilhaVazia(&tempLimpa)) {
113             int vVerifica = desempilhar(&tempLimpa);
114             if (vVerifica == atual) {
115                 duplicado = 1;
116             }
117             empilhar(&tempVerificadora, vVerifica);
118         }
119         while (!pilhaVazia(&tempVerificadora)) {
120             empilhar(&tempLimpa, desempilhar(&tempVerificadora));
121         }
122         if (!duplicado) {
123             empilhar(&tempLimpa, atual);
124         }
125     }
126     while (!pilhaVazia(&tempLimpa)) {
127         empilhar(estrutura, desempilhar(&tempLimpa));
128     }
129 }

```